

Learning Gradients of Convex Functions with Monotone Gradient Networks

Benjamin Deporte : benjamin.deporte@ens-paris-saclay.fr
Liam Ludington : whfludington@gmail.com

March 2025

Abstract

This report presents my work regarding the article [2] "Learning Gradients of Convex Functions with Monotone Gradient Networks", as part of the final project of the class "Generative Modeling" of the MVA class 2024-2025.

The code is available at : https://github.com/BenjaminDeporte/MVA_GenAI_FP_Learning_Gradients

1 Introduction

Convex optimization is a general-purpose field that is present in many applications. More important than the objective function involved in convex optimization formulations, is the *gradient* of the objective function. This gradient is naturally involved in numerical approximation methods, inverse problems, and optimal transport.

The article describes a simple but powerful method to learn a monotone gradient function via a deep learning network.

The approach is to design a network architecture such that the function represented by the network is mechanically a monotone gradient, without need to further optimize or constrain the training. This "gradient by design" is made possible by ensuring that the Jacobian of the network is actually positive definite, which is the second-order convexity condition for a function.

2 Optimal Transport Survival Kit

We reproduce here some key Optimal Transport results from [1]. The Brenier theorem is at the heart of the approach of the article.

2.1 General Measures

We consider here the set of Radon measures $\mathcal{M}(\mathcal{X})$ over a set $\mathcal{X}$, and the set $\mathcal{C}(\mathcal{X})$ of continuous functions over $\mathcal{X}$. $\mathcal{M}(\mathcal{X})$ is the set of measures α over $\mathcal{X}$ that verify: $\forall f \in \mathcal{C}(\mathcal{X}), \int_{\mathcal{X}} f(x) d\alpha(x) \in \mathbb{R}$.

We note $\mathcal{M}_+(\mathcal{X})$ the subset of $\mathcal{M}(\mathcal{X})$ composed of positive measures, and $\mathcal{M}_+^1(\mathcal{X})$ the subset of probability measures (ie $\int_{\mathcal{X}} f(x) d\alpha(x) = 1$).

If α has a density wrt the Lebesgue measure, ie $d\alpha(x) = \rho_\alpha(x)$, then $\forall f \in \mathcal{C}(\mathbb{R}^d), \int_{\mathbb{R}^d} f(x) d\alpha(x) = \int_{\mathbb{R}^d} f(x) \rho_\alpha(x) dx$.

2.2 Push Forward

Let $\mathcal{X}$ and $\mathcal{Y}$ be two measurable sets, $T : \mathcal{X} \rightarrow \mathcal{Y}$ continuous.

For $\alpha \in \mathcal{M}(\mathcal{X})$, the **push forward measure** $\beta = T_\# \alpha \in \mathcal{M}(\mathcal{Y})$ satisfies :

$$\forall h \in \mathcal{C}(\mathcal{Y}), \int_{\mathcal{Y}} h(y) d\beta(y) = \int_{\mathcal{X}} h(T(y)) d\alpha(x)$$

or equivalently :

$$\begin{aligned} \forall B \in \mathcal{Y} \text{ measurable}, \beta(B) &= \alpha(\{x \in \mathcal{X} \text{ s.t. } T(x) \in B\}) \\ &= \alpha(T^{-1}(B)) \end{aligned}$$

The application T is often referred to as the *transport map* from distribution α to distribution β .

2.3 Monge problem

The **Monge problem** consists in finding a continuous map T from $\mathcal{X}$ to $\mathcal{Y}$ that transports (ie *pushes forward*) a measure α to a measure β , minimizing a transport cost $c(x, y)$.

Formally :

$$\begin{aligned} &\text{for } \alpha \in \mathcal{M}(\mathcal{X}), \beta \in \mathcal{M}(\mathcal{Y}), \\ &\text{for } c : \mathcal{X} \times \mathcal{Y} \rightarrow \mathbb{R} \\ &\text{find } T^* : \mathcal{X} \rightarrow \mathcal{Y} \text{ continuous st} \\ &T^* = \operatorname{argmin}_T \left\{ \int_{\mathcal{X}} c(x, T(x)) d\alpha(x), \text{ st } T_\# \alpha = \beta \right\} \end{aligned}$$

This formulation may or may not have a solution, as it requires a point-to-point transport.

2.4 Kantorovitch relaxation

Kantorovitch symetrizes the problem and relaxes it by allowing transport between distributions, and not only between points.

We note the set of joint probability measures over $\mathcal{X} \times \mathcal{Y}$ with marginales α over $\mathcal{X}$ and β over $\mathcal{Y}$ by:

$$\mathcal{U}(\alpha, \beta) = \{\pi \in \mathcal{M}_+^1(\mathcal{X} \times \mathcal{Y}) \mid \text{st} P_{\mathcal{X}\#}\pi = \alpha, P_{\mathcal{Y}\#}\pi = \beta\}$$

The Kantorovitch formulation is :

$$\mathcal{L}_c(\alpha, \beta) = \min_{\pi \in \mathcal{U}(\alpha, \beta)} \int_{\mathcal{X} \times \mathcal{Y}} c(x, y) d\pi(x, y)$$

The problem is then to find a joint probability distribution, subject to two marginals, minimizing a cost function.

2.5 Brenier Theorem

The Brenier's theorem is an important result for $\mathcal{X} = \mathcal{Y} = \mathbb{R}^d$, when c is the squared Euclidean distance $c(x, y) = \|x - y\|^2$.

The Kantorovitch formulation is then:

$$\min_{\pi \in \mathcal{U}(\alpha, \beta)} \int_{\mathbb{R}^d \times \mathbb{R}^d} \|x - y\|^2 d\pi(x, y)$$

The theorem states :

- **if** at least one of the measures (say here α) is absolutely continuous wrt Lebesgue measure (ie has a density $\rho_\alpha : d\alpha(x) = \rho_\alpha(x)$),
- **then**
 - the optimal π **exists, is unique**, has for support $(x, T(x))$ for $x \in \text{supp}(\alpha)$,
 - $T : \mathbb{R}^d \longrightarrow \mathbb{R}^d$ is the "Monge map" ($\pi = (\text{id}, T)_\# \alpha$):

$$\forall h \in \mathcal{C}(\mathcal{X} \times \mathcal{Y}), \int_{\mathcal{X} \times \mathcal{Y}} h(x, y) d\pi(x, y) = \int_{\mathcal{X}} h(x, T(x)) d\alpha(x)$$

- $T(x) = \nabla \phi(x)$ with ϕ **unique** (up to an additive constant), **convex**, with $(\nabla \phi)_\# \alpha = \beta$

To summarize, when the transport cost is the Euclidean distance between points, then there is a unique optimal transport map that is the gradient flow of a convex function.

The next paragraph describes how to characterize such a gradient flow.

3 Problem statement

The idea is to learn a transport map as a gradient of a convex function, by designing a neural network that will structurally learn such a function.

We review first some results on convexity.

3.1 Convexity results

The reference for the results is [3]. **Proofs are in the appendix.**

- **convexity first order condition** - Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ differentiable. Then f is convex iff its gradient ∇f is monotone : $\forall x, y, \langle \nabla f(x) - \nabla f(y), x - y \rangle \geq 0$
- **convexity second order result** - Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ twice differentiable. Then f is convex iff $\nabla^2 f(x) \succeq 0$ (where $\nabla^2 f(x)$ is the Hessian of f).

3.2 Application

Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ convex, twice differentiable. Let $g(x) = \nabla f(x)$. Then:

$$f \text{ convex} \iff \nabla^2 f(x) = J_g(x) \succeq 0, \forall x \in \mathbb{R}^n$$

where $J_g(x)$ is the Jacobian of g taken in x .

If $g(x) = g_\theta(x)$ is a neural network, then

$$g = \nabla f \iff J_{g_\theta}(x) \succeq 0, \forall x \in \mathbb{R}^n$$

The network g_θ represents a gradient field -and an optimal transport map for the L2 cost- iff its Jacobian is positive semi definite everywhere.

4 Proposed approaches

The paper proposes two network architectures, such that the network function has a PSD Jacobian by design.

4.1 Cascaded Network: C-MGN

The first model is **C-MGN** :

$$\begin{aligned} z_0 &= Wx + b_0 \text{ first layer} \\ z_l &= Wx + \sigma_l(z_{l-1}) + b_l \text{ next layers} \\ \text{C-MGN}(x) &= W^T \sigma_L(z_{L-1}) + V^T Vx + b_L \end{aligned}$$

The weights matrix W is shared between all layers.

Proposition 1:

if $\forall l, \sigma_l(\cdot)$ is a differentiable, element-wise activation function that is monotonically increasing, then $\forall x, J_{C-MGN}(x) \succeq 0$

We calculate the Jacobian of $C-MGN(x) = W^T \sigma_L(z_{L-1})(x) + V^T Vx + b_L$.

First,

$$\begin{aligned}
J_{\sigma_L \circ z_{L-1}}(x) &= J_{\sigma_L}(z_{L-1})(x) \cdot J_{z_{L-1}}(x) \\
\text{with } z_{L-1}(x) &= Wx + \sigma_{L-1}(z_{L-2}) + b_{L-1} \\
J_{z_{L-1}}(x) &= W + J_{\sigma_{L-1} \circ z_{L-2}}(x) \\
J_{\sigma_L \circ z_{L-1}}(x) &= J_{\sigma_L}(z_{L-1}) \cdot (W + J_{\sigma_{L-1} \circ z_{L-2}}(x)) \\
&= J_{\sigma_L}(z_{L-1})W + J_{\sigma_L}(z_{L-1}) \cdot J_{\sigma_{L-1} \circ z_{L-2}}(x) \\
&= J_{\sigma_L}(z_{L-1})W + J_{\sigma_L}(z_{L-1}) (J_{\sigma_{L-1}}(z_{L-2})(W + J_{\sigma_{L-2} \circ z_{L-3}}(x))) \\
&= \dots \\
&= (J_{\sigma_L}(z_{L-1}) + J_{\sigma_L}(z_{L-1})J_{\sigma_{L-1}}(z_{L-2}) + \dots + J_{\sigma_L}(z_{L-1}) \times \dots \times J_{\sigma_1}(z_0)) W \\
&= \left(\sum_{l=1}^L \prod_{i=l}^L J_{\sigma_i}(z_{i-1}) \right) W
\end{aligned}$$

So :

$$J_{C-MGN}(x) = W^T \left(\sum_{l=1}^L \prod_{i=l}^L J_{\sigma_i}(z_{i-1}) \right) W + V^T V$$

The activation functions are $\sigma_i(\cdot)$ are monotonically increasing element wise :

$$\begin{aligned}
\sigma_i(x) &= \begin{pmatrix} \sigma_i(x_1) \\ \dots \\ \sigma_i(x_p) \end{pmatrix} \\
J_{\sigma_i}(x) &= \begin{pmatrix} \frac{\partial \sigma_i}{\partial x_1} & 0 & \dots & 0 \\ 0 & \frac{\partial \sigma_i}{\partial x_2} & \dots & 0 \\ 0 & \dots & \dots & 0 \\ 0 & \dots & \dots & \frac{\partial \sigma_i}{\partial x_p} \end{pmatrix} \succeq 0
\end{aligned}$$

$$\text{So } \sum_{l=1}^L \prod_{i=l}^L J_{\sigma_i}(z_{i-1}) \succeq 0.$$

Finally $J_{C-MGN}(x) = W^T A W + V^T V$ with $A \succeq 0$.

So $J_{C-MGN}(x) \succeq 0$.

Which proves that **C-MGN is the gradient of a convex function.**

4.2 Modular Network : M-MGN

The **M-MGN** is another network architecture, that produces by design a function with a PSD Jacobian.

The model is:

$$z_k = W_k x + b_k$$

$$M - MGN(x) = a + V^T V x + \sum_{k=1}^K s_k(z_k) W_k^T \sigma_k(z_k)$$

where $s_k(\cdot)$ is a *scalar* function $s_k : \mathbb{R}^n \rightarrow \mathbb{R}$ that is **convex, twice differentiable, non-negative**, and that verifies:

$$\sigma_k(\cdot) = \nabla s_k(\cdot)$$

Then, if we write:

$$g_k(x) = s_k(z_k) W_k^T \sigma_k(z_k)(x)$$

The Jacobian writes:

$$\begin{aligned} J_{g_k}(x) &= J_{s_k(z_k)} W_k^T \sigma_k(z_k) + s_k(z_k) W_k^T J_{\sigma_k(z_k)}(x) \\ &= J_{s_k(z_k)} W_k^T \sigma_k(z_k) + s_k(z_k) W_k^T J_{\sigma_k(z_k)} J_{z_k}(x) \end{aligned}$$

We have $z_k = W_k x + b_k$, so $J_{z_k} = W_k$. And also:

$$J_{s_k} = \begin{pmatrix} \nabla s_k|_1^T \\ \nabla s_k|_2^T \\ \dots \\ \nabla s_k|_n^T \end{pmatrix} = \begin{pmatrix} \sigma_1^T \\ \sigma_2^T \\ \dots \\ \sigma_n^T \end{pmatrix} = \sigma_k^T$$

Finally:

$$\begin{aligned} J_{g_k}(x) &= J_{s_k(z_k)} W_k^T \sigma_k(z_k) + s_k(z_k) W_k^T J_{\sigma_k(z_k)} W_k \\ J_{s_k(z_k)} &= J_{s_k}(z_k) \cdot J_{z_k} = \sigma_k(z_k)^T W_k \\ J_{g_k}(x) &= (\sigma_k(z_k)^T W_k)(W_k^T \sigma_k(z_k)) + s_k(z_k) W_k^T J_{\sigma_k(z_k)} W_k \end{aligned}$$

Finally:

$$J_{M-MGN}(x) = V^T V + \sum_{k=1}^K (s_k(z_k) W_k^T J_{\sigma_k(z_k)} W_k + (W_k^T \sigma_k(z_k))(W_k^T \sigma_k(z_k))^T)$$

$s_k(\cdot)$ is convex, so $J_{\sigma_k} \succeq 0$. $s_k(\cdot)$ is also scalar non-negative, so the first term $s_k(z_k) W_k^T J_{\sigma_k(z_k)} W_k$ is PSD. So are $(\sigma_k(z_k)^T W_k)(W_k^T \sigma_k(z_k))$ and $V^T V$. Finally, $J_{M-MGN} \succeq 0$.

So M-MGN is the gradient of a convex function.

4.3 Discussion

We note that C-MGN is a deep network architecture, whereas M-MGN is a parallel architecture. C-MGN will take as an hyperparameter the number of layers (placed sequentially), where as J-MGN will take as an hyperparameter the number of modules (placed in parallel).

The conditions on σ_l (for C-MGN) allow many activation functions, such as tanh, sigmoid, softplus. For M-MGN, the condition $s_k(x) \geq 0$ is not restrictive as, for example, taking $s_k(x) = \log \cosh(x_1) + \dots + \log \cosh(x_n)$ yields $g_k(x) = \tanh(x)$. The code for M-MGN uses log cosh for s_k , and tanh for activations.

5 Experiments

The experiments are easier to follow in the attached notebook. There are two toy cases and a domain adaptation experiment.

5.1 Toy case 1 : Gradient field

The convex function is $f : \mathbb{R}^2 \rightarrow \mathbb{R}$:

$$f(x_1, x_2) = x_1^4 + \frac{x_1^2}{2} + \frac{x_1 x_2}{2} + \frac{3x_2^2}{2} - \frac{x_2^3}{3} \quad (1)$$

The gradient field to learn, ie the transport map, is:

$$T(x_1, x_2) = \begin{pmatrix} 4x_1^3 + \frac{1}{2}x_2 + x_1 \\ 3x_2 - x_2^2 + \frac{1}{2}x_1 \end{pmatrix} \quad (2)$$

Both a C-MGN and a M-MGN networks are trained, with a L1 loss between the model computations and the target fields. Both architectures train well.

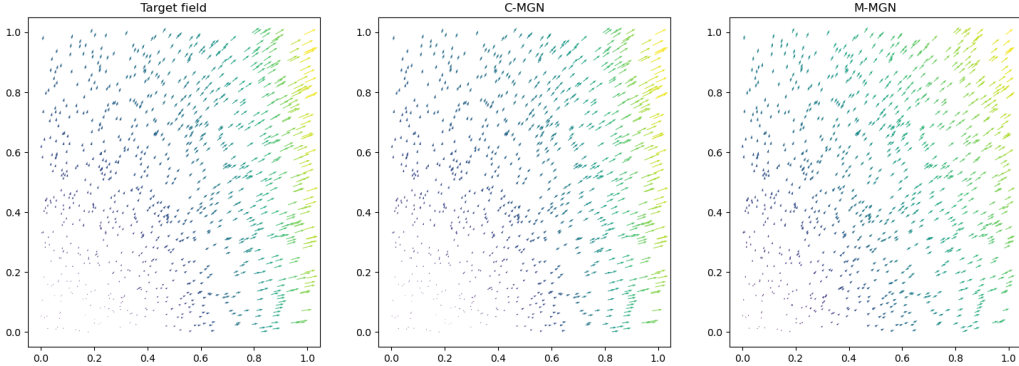


Figure 1: Gradient field learning by C-MGN and M-MGN

5.2 Toy case 2 : Optimal Coupling between Gaussians

Here, the two architectures learn a coupling between a source Gaussian ($\mathcal{N}(0, 2 * \text{Id}_2)$) and a target Gaussian.

Two different losses are used, that have closed form solutions for two Gaussians : the Wasserstein-2 distance, and the KL-divergence. Recall that the W2 distance and KL-divergence between two Gaussians $\mathcal{N}_1(\mu_1, \Sigma_1)$ and $\mathcal{N}_2(\mu_2, \Sigma_2)$ are:

$$\mathcal{W}(\mathcal{N}_1, \mathcal{N}_2) = \|\mu_1 - \mu_2\|_2^2 + \text{Trace} \left(\Sigma_1 + \Sigma_2 - 2 * (\Sigma_2^{\frac{1}{2}} \Sigma_1 \Sigma_2^{\frac{1}{2}})^{\frac{1}{2}} \right)$$

$$\mathbb{KL}(\mathcal{N}_1 || \mathcal{N}_2) = \frac{1}{2} \left(\text{Trace}(\Sigma_2^{-1} \Sigma_1) - k + (\mu_2 - \mu_1)^T \Sigma_2^{-1} (\mu_2 - \mu_1) + \ln \left(\frac{|\Sigma_2|}{|\Sigma_1|} \right) \right)$$

where k is the dimensionnality of the two Gaussians.

Overall, the C-MGN trains faster and easier than the M-MGN. The training of the M-MGN with the KL-divergence is difficult.

5.3 Domain adaptation - color transport

Here, the color distribution of a daytime image is mapped to the color distribution of a target image (either "sunset" or "night"), both distributions approximated by a 3D Gaussian. A C-MGN with a Wasserstein-2 loss is used.



Figure 2: Day to Sunset



Figure 3: Day to Night

Appendices

A Proofs

A.1 Result 1 - Convexity

Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ differentiable. Then f is convex iff its gradient ∇f is monotone : $\forall x, y \in \text{dom} f, \langle \nabla f(x) - \nabla f(y), x - y \rangle \geq 0$.

Proof

f is convex iff $\forall x, y \in \mathbb{R}^d, f(y) \geq f(x) + \langle \nabla f(x), y - x \rangle$. Substituting x and y , we also have $f(x) \geq f(y) + \langle \nabla f(y), x - y \rangle$. Adding the two equations, we get $0 \geq \langle y - x, \nabla f(x) - \nabla f(y) \rangle$, or $\langle \nabla f(y) - \nabla f(x), y - x \rangle \geq 0$

A.2 Result 2 - Convexity

Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ twice differentiable. Then f is convex iff $\nabla^2 f(x) \succeq 0$ (where $\nabla^2 f(x)$ is the Hessian of f).

Proof

First, assume $n = 1$. $f : \mathbb{R} \rightarrow \mathbb{R}$ twice differentiable.

- if f is convex, then the first order condition writes

$$\begin{aligned} \forall x, y \in \text{dom} f, y > x &\implies f'(x)(y - x) \leq f(y) - f(x) \leq f'(y)(y - x) \\ &\implies \frac{f'(y) - f'(x)}{y - x} \geq 0 \\ &\implies f''(x) = \lim_{y \rightarrow x} \frac{f'(y) - f'(x)}{y - x} \geq 0 \end{aligned}$$

- conversely, if $\forall z \in \text{dom} f, f''(z) \geq 0$. Then :

$$\begin{aligned} \forall x, y \in \text{dom} f, x < y &\implies 0 \leq \int_x^y f''(z)(y - z) dz \\ &= f'(z)(y - z)|_x^y + \int_x^y f'(z) dz \\ &= -f'(x)(y - x) + f(y) - f(x) \end{aligned}$$

which proves f is convex.

Second, for $n \geq 2$, f is convex iff it is convex on all lines, ie $g(t) = f(x_0 + tv)$ convex in $t \forall x_0 \in \text{dom} f, \forall v$. So f is convex iff

$$\begin{aligned} g''(t) &= v^T \nabla^2 f(x_0 + tv) v \geq 0, \forall x_0 \in \text{dom} f, \forall v \in \mathbb{R}^n, \forall t \text{ st } x_0 + tv \in \text{dom} f \\ &\iff \nabla^2 f(x) \succeq 0 \end{aligned}$$

A.3 M-CGN

We calculate the Jacobian of $C - MGN(x) = W^T \sigma_L(z_{L-1})(x) + V^T Vx + b_L$.

First,

$$\begin{aligned}
J_{\sigma_L \circ z_{L-1}}(x) &= J_{\sigma_L}(z_{L-1})(x) \cdot J_{z_{L-1}}(x) \\
\text{with } z_{L-1}(x) &= Wx + \sigma_{L-1}(z_{L-2}) + b_{L-1} \\
J_{z_{L-1}}(x) &= W + J_{\sigma_{L-1} \circ z_{L-2}}(x) \\
J_{\sigma_L \circ z_{L-1}}(x) &= J_{\sigma_L}(z_{L-1}) \cdot (W + J_{\sigma_{L-1} \circ z_{L-2}}(x)) \\
&= J_{\sigma_L}(z_{L-1})W + J_{\sigma_L}(z_{L-1}) \cdot J_{\sigma_{L-1} \circ z_{L-2}}(x) \\
&= J_{\sigma_L}(z_{L-1})W + J_{\sigma_L}(z_{L-1}) (J_{\sigma_{L-1}}(z_{L-2})(W + J_{\sigma_{L-2} \circ z_{L-3}}(x))) \\
&= \dots \\
&= (J_{\sigma_L}(z_{L-1}) + J_{\sigma_L}(z_{L-1})J_{\sigma_{L-1}}(z_{L-2}) + \dots + J_{\sigma_L}(z_{L-1}) \times \dots \times J_{\sigma_1}(z_0)) W \\
&= \left(\sum_{l=1}^L \prod_{i=l}^L J_{\sigma_i}(z_{i-1}) \right) W
\end{aligned}$$

So :

$$J_{C-MGN}(x) = W^T \left(\sum_{l=1}^L \prod_{i=l}^L J_{\sigma_i}(z_{i-1}) \right) W + V^T V$$

A.4 M-MGN

The model is:

$$\begin{aligned}
z_k &= W_k x + b_k \\
M - MGN(x) &= a + V^T Vx + \sum_{k=1}^K s_k(z_k) W_k^T \sigma_k(z_k)
\end{aligned}$$

where $s_k(\cdot)$ is a *scalar* function $s_k : \mathbb{R}^n \longrightarrow \mathbb{R}$ that is **convex**, **twice differentiable**, **non-negative**, and that verifies:

$$\sigma_k(\cdot) = \nabla s_k(\cdot)$$

Then, if we write:

$$g_k(x) = s_k(z_k) W_k^T \sigma_k(z_k)(x)$$

The Jacobian writes:

$$\begin{aligned} J_{g_k}(x) &= J_{s_k(z_k)} W_k^T \cdot \sigma_k(z_k) + s_k(z_k) W_k^T J_{\sigma_k(z_k)}(x) \\ &= J_{s_k(z_k)} W_k^T \sigma_k(z_k) + s_k(z_k) W_k^T J_{\sigma_k(z_k)} J_{z_k}(x) \end{aligned}$$

We have $z_k = W_k x + b_k$, so $J_{z_k} = W_k$. And also:

$$J_{s_k} = \begin{pmatrix} \nabla s_k|_1^T \\ \nabla s_k|_2^T \\ \dots \\ \nabla s_k|_n^T \end{pmatrix} = \begin{pmatrix} \sigma_1^T \\ \sigma_2^T \\ \dots \\ \sigma_n^T \end{pmatrix} = \sigma_k^T$$

Finally:

$$\begin{aligned} J_{g_k}(x) &= J_{s_k(z_k)} W_k^T \sigma_k(z_k) + s_k(z_k) W_k^T J_{\sigma_k(z_k)} W_k \\ J_{s_k(z_k)} &= J_{s_k(z_k)} \cdot J_{z_k} = \sigma_k(z_k)^T W_k \\ J_{g_k}(x) &= (\sigma_k(z_k)^T W_k)(W_k^T \sigma_k(z_k)) + s_k(z_k) W_k^T J_{\sigma_k(z_k)} W_k \end{aligned}$$

Finally:

$$J_{M-MGN}(x) = V^T V + \sum_{k=1}^K (s_k(z_k) W_k^T J_{\sigma_k(z_k)} W_k + (W_k^T \sigma_k(z_k))(W_k^T \sigma_k(z_k))^T)$$

References

- [1] G. Peyré and M. Cuturi, *Computational Optimal Transport*, arXiv:1803.00567, Mar. 2020. DOI: [10.48550/arXiv.1803.00567](https://arxiv.org/abs/1803.00567). [Online]. Available: <http://arxiv.org/abs/1803.00567> (visited on 03/17/2025).
- [2] S. Chaudhari, S. Pranav, and J. M. F. Moura, *Learning Gradients of Convex Functions with Monotone Gradient Networks*, arXiv:2301.10862, Mar. 2023. DOI: [10.48550/arXiv.2301.10862](https://arxiv.org/abs/2301.10862). [Online]. Available: <http://arxiv.org/abs/2301.10862> (visited on 03/17/2025).
- [3] *Convex Optimization – Boyd and Vandenberghe*. [Online]. Available: <https://stanford.edu/~boyd/cvxbook/> (visited on 03/17/2025).